22 · Custom Directives

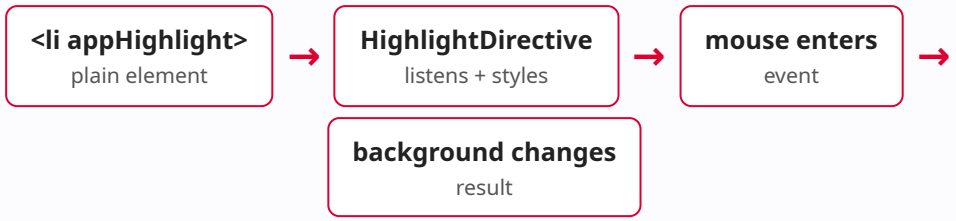
Intermediate Angular learning notes

a. Definition

A **custom directive** adds reusable behaviour to any existing element, without a template of its own. You create one with `@Directive` and a CSS-attribute **selector** such as `[appHighlight]`.

The `host` property connects the directive to the element it sits on: `'(mouseenter)'` listens to events and `'[style.color]'` binds styles. Directives can also have `input()`s and `output()`s like components.

b. Diagram



The directive attaches behaviour to a normal element

c. Syntax

```
@Directive({
  selector: '[appHighlight]',
  host: { '(mouseenter)': 'onEnter()', '[style.background-color]': 'color()' },
})
export class HighlightDirective {
  color = input('yellow', { alias: 'appHighlight' });
}

<li appHighlight="lightgreen">Idli</li>
```

d. Example Program

Programs/22_Custom_Directives/highlight.directive.ts

```
import { Component, Directive, input, output, signal } from '@angular/core';

// An attribute directive: adds hover highlighting to ANY element
@Directive({
  selector: '[appHighlight]',
  host: {
    '(mouseenter)': 'enter()',
    '(mouseleave)': 'leave()',
    // style binding on the host element
    '[style.background-color]': 'active() ? color() : null',
  },
})
export class HighlightDirective {
  color = input('yellow', { alias: 'appHighlight' }); // <li appHighlight="lightgreen">
  hoverChange = output<boolean>();                     // tell the parent about hover
```

```
  active = signal(false);

  enter() {
    this.active.set(true);
    this.hoverChange.emit(true);
  }
  leave() {
    this.active.set(false);
    this.hoverChange.emit(false);
  }
}

@Component({
  selector: 'app-root',
  imports: [HighlightDirective],
  template: `
    <ul>
      @for (item of items; track item) {
        <li appHighlight="lightgreen" (hoverChange)="onHover(item, $event)">{{ item }}</li>
      }
    </ul>
    <p>Hovered: {{ hovered() || 'nothing' }}</p>
  `,
})
export class AppComponent {
  items = ['Idli', 'Filter Coffee', 'Vada'];
  hovered = signal('');

  onHover(item: string, isOver: boolean) {
    this.hovered.set(isOver ? item : '');
  }
}
```

e. Output

Rendered in the browser (text on screen, after the UI steps below):

1. Move the mouse over **Filter Coffee**.

```
Idli
Filter Coffee
Vada
Hovered: Filter Coffee
```

f. Real-World Example in Tamil (தமிழ் உதாரணம்)

பள்ளியில் ஆசிரியர் ஒரு மஞ்சள் மார்க்கர் (highlighter) பேனாவை வைத்திருக்கிறார். எந்தப் புத்தகத்தின் எந்த வரியிலும் அதைப் பயன்படுத்தி முக்கியமான இடத்தை அடையாளம் காட்டலாம். வரியின் உள்ளடக்கம் மாறுவதில்லை; அதன் தோற்றம் மட்டும் மாறுகிறது.

Custom directive அந்த மார்க்கர் பேனா: `appHighlight` என்று எந்த element மீதும் எழுதிவிட்டால் போதும், அதில் மவுஸ் வந்தால் நிறம் மாறும். ஒரு முறை எழுதி எங்கும் மறுபடி பயன்படுத்தலாம்.

g. Common Mistakes

- Using a selector without brackets (`appHighlight` instead of `[appHighlight]`) so it matches an element name.
- Not prefixing the selector (use `app` or your own prefix) and clashing with HTML attributes.
- Manipulating `nativeElement.style` directly when a host binding would do.
- Forgetting to import the directive in the component that uses it.

h. Interview Questions

Q1. What is a directive?
A class that adds behaviour to elements. Types: components, structural and attribute directives.

Q2. How is an attribute directive created?
With `@Directive({ selector: '[name]' })`, using `host` bindings/listeners or `inject(ElementRef)`.

Q3. What does `host` do?
It binds properties, attributes, styles and events on the element that hosts the directive.

Q4. Directive vs component?
A component is a directive with a template; a directive has no template.

i. Practice Exercises

1. Create `appAutoFocus` which focuses its element when the page loads.
2. Write `appUppercase` that converts typed text in an input to capital letters.
3. Build `appTooltip="text"` that shows the text as the element's `title` attribute.